SimpleStoreServiceOrchestration

System Description

Abstract
This document describes the SimpleStoreServiceOrchestration core system of the Eclipse Arrowhead 5.2.2 framework, as implemented in Java Spring in `ah5-core-java-spring/serviceorchestration-simple`, including the local extensions of this workspace.

Contents

1 Overview **3**
1.1 Significant Prior Art . . . 4
1.2 How This System Is Meant to Be Used . . . 4
1.3 System functionalities and properties . . . 4
1.4 Important Delimitations . . . 5

2 Services **6**
2.1 Produced services . . . 6
2.2 Consumed services . . . 6

3 Security **7**
3.1 Security model . . . 7

4 References **7**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the SimpleStoreServiceOrchestration system. SimpleStoreServiceOrchestration orchestrates from a static store: an operator records which service instance each consumer shall use, with a priority, and the orchestrator returns these rules on request. It offers the same `serviceOrchestration` service as DynamicServiceOrchestration, plus management of the store.



Figure 1: SysML-style block view of SimpleStoreServiceOrchestration with its produced and consumed services.

The rest of this document is organized as follows. In Section 1.1, we reference major prior art capabilities of the system. In Section 1.2, we describe the intended usage of the system. In Section 1.3, we describe fundamental properties provided by the system. In Section 1.4, we describe delimitations of capabilities of the system. In Section 2, we describe the abstract service functions consumed or produced by the system. In Section 3, we describe the security capabilities of the system.

1.1 Significant Prior Art

Successor of the store-based orchestration of the Arrowhead 4.x Orchestrator. In Arrowhead 5 it is a separate, deliberately simple system.

1.2 How This System Is Meant to Be Used

An operator creates store entries with `serviceOrchestrationSimpleStoreManagement.create`. A consumer pulls with the service definition (and optional preferred providers and flags `MATCHMAKING`, `ONLY_PREFERRED`) and receives the instance identifiers in priority order. It then looks up the interfaces of the chosen instance in the ServiceRegistry. Under `service.discovery.policy=restricted`, the default, that lookup only works when the consumer is listed in `service.discovery.direct.access`; otherwise the instance is silently missing from the result. In the test clouds it runs alongside DynamicServiceOrchestration; Java consumers pin the orchestrator by provider name.

1.3 System functionalities and properties

1.3.1 Functional properties of the system

- Pull orchestration from the store in priority order, with preferred providers and matchmaking.
- Push orchestration with subscriptions and triggers, as DynamicServiceOrchestration.
- Management of store entries: create, query, change priorities, remove.
- Orchestration history.

1.3.2 Configuration of system properties

The system is configured with Spring properties (`application.properties`, overridden by environment variables in the container deployment). The properties that shape its behaviour are:

Property	Default	Meaning
`server.port`	8442	HTTPS port (port specification). The upstream image still carries 8465; an image built from this tree does not.
`authentication.policy`	declared	Test cloud: `outsourced`.
`arrowhead.security.profile`	secure	`development` selects the isolated development exception; never set in a deployment.
`management.policy` / `management.whitelist`	authorization	Test cloud: whitelist LocalCloudManager (PDE now uses DSO).
`enable.blacklist.filter`	false	Blacklist filter of requesters (test cloud: true).
`orchestration.history.max.age`	15	Days the history is kept.
`cleaner.job.interval`	30000	Cleaner interval in ms.
`push.orchestration.max.thread`	5	Push worker threads.
`normalization.mode`	extended	Name normalization.
`mqtt.api.enabled`	false	Offer the MQTT interfaces.
`sysmonitor.enabled`	false	Publish native sysMonitor observations (test cloud: true).
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate (local).

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; in the test clouds they are per-cloud secrets mounted as files.

1.3.3 Data stored by the system

Orchestration store entries (consumer, service definition, instance, priority, creator), push subscriptions, orchestration jobs and log entries (database `ah_serviceorchestration_simple`).

1.3.4 Non functional properties

A pull is a single database query; no other system is called during orchestration, so it works while Consumer Authorization is unavailable.

1.3.5 Stateful or stateless

Stateful: store entries, subscriptions and history are persistent.

1.4 Important Delimitations

The store is not checked against the ServiceRegistry or ConsumerAuthorization; a rule can point to an expired instance or an unauthorized provider. Results contain no interfaces, metadata or tokens. Only the flags `MATCHMAKING` and `ONLY_PREFERRED` are supported.

2 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

2.1 Produced services

Service	Version	Interfaces	SD	IDD
`serviceOrchestration`	1.0.0	HTTP(S), MQTT(S)	[1]	[2], [3]
`serviceOrchestrationSimpleStoreManagement`	1.0.0	HTTP(S), MQTT(S)	[4]	[5], [6]
`serviceOrchestrationPushManagement`	1.0.0	HTTP(S), MQTT(S)	[7]	[8], [9]
`serviceOrchestrationHistoryManagement`	1.0.0	HTTP(S), MQTT(S)	[10]	[11], [12]
`generalManagement`	1.0.0	HTTP(S), MQTT(S)	[13]	[14], [15]
`sysMonitor`	1.0.0	AMQP(S)	[16]	[17]

The HTTP interfaces are always offered, the MQTT interfaces with `mqtt.api.enabled=true` and the AMQP `sysMonitor` interface with `sysmonitor.enabled=true`. Upstream 5.2.2 also answers `GET <base>/monitor/echo` on some systems; this endpoint is not registered as a service.

2.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	serviceDiscovery	register, revoke, lookup	Registration; lookup of core services.	always
Authentication	identity	identity-login, identity-verify	Own token; verification of requester tokens.	`authentication.policy=outsourced`
Consumer Authorization	authorization	verify	Management filter decisions.	`management.policy=authorization`
Blacklist	blacklistDiscovery	check	Blacklist filter.	`enable.blacklist.filter=true`
Subscribers	notify interface	HTTP(S) request or MQTT(S) publish	Push orchestration results.	subscriptions
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status	Own observations.	`sysmonitor.enabled=true`

Except for the bootstrap services (`http.collector.bootstrap.services`), which are looked up in the ServiceRegistry, the system reaches other systems' services through DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`, results cached for `http.collector.orchestration.ttl` = 300 s); every such call needs a ConsumerAuthorization grant.

3 Security

The default is the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The Arrowhead-test-cloud-java deployment runs the secure mode.

The packaged `application.properties` is a development sample that disables TLS and declares identities. The system refuses to start with it, or with any other effective configuration that sets `server.ssl.enabled=false`, `authentication.policy=declared`, `service.authorization.enabled=false` or, while monitoring is on, `sysmonitor.ssl.enabled=false`. The check runs before the system accepts a connection. Only an explicit `arrowhead.security.profile=development` lets such a system start, for an isolated test, and it then logs a non-conformance warning.

3.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates (`server.ssl.client-auth=need`); MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the test cloud. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** by the authentication policy: `certificate` (client certificate CN), `outsourced` (identity token verified with Authentication), `internal` (Authentication only) or `declared` (insecure). In this workspace an identity token is accepted only together with the client certificate of the system it was issued to (token-to-certificate binding), and every client certificate is checked against the cloud CA's CRL when `arrowhead.ssl.crl-file` is set (fail closed).
- **Authorization of produced services:** management services are protected by the management filter (`management.policy`: `SYSOP_ONLY`, `WHITELIST` or `AUTHORIZATION` via ConsumerAuthorization); an optional blacklist filter refuses blacklisted requesters. Sysop identities bypass the management filter.
- Because the store bypasses ConsumerAuthorization, operators must keep store entries consistent with the authorization policies; providers should still check the consumer's identity. Three combinations with the ServiceRegistry discovery policy are intended: (1) DynamicServiceOrchestration for ordinary consumers, the recommended default, where no consumer needs discovery access; (2) `restricted` with each SimpleStore consumer named in `service.discovery.direct.access`, which also lets those consumers discover every other service, so a provider must still check them; (3) `open`, only for an isolated cloud where every system may discover every service. A consumer that holds the access details in its local configuration needs no lookup and no discovery access.

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

4 References

[1] J. Delsing, "serviceOrchestration – Service Description (SimpleStoreServiceOrchestration)," Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[2] ——, "serviceOrchestration HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration)," Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-HTTP-IDD.pdf.

[3] ——, "serviceOrchestration MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration)," Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestration-MQTT-IDD.pdf.

[4] ——, “serviceOrchestrationSimpleStoreManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-SD.pdf.

[5] ——, “serviceOrchestrationSimpleStoreManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-HTTP-IDD.pdf.

[6] ——, “serviceOrchestrationSimpleStoreManagement MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-MQTT-IDD.pdf.

[7] ——, “serviceOrchestrationPushManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-SD.pdf.

[8] ——, “serviceOrchestrationPushManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-HTTP-IDD.pdf.

[9] ——, “serviceOrchestrationPushManagement MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationPushManagement-MQTT-IDD.pdf.

[10] ——, “serviceOrchestrationHistoryManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-SD.pdf.

[11] ——, “serviceOrchestrationHistoryManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-HTTP-IDD.pdf.

[12] ——, “serviceOrchestrationHistoryManagement MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationHistoryManagement-MQTT-IDD.pdf.

[13] ——, “generalManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-SD.pdf.

[14] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-HTTP-IDD.pdf.

[15] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-generalManagement-MQTT-IDD.pdf.

[16] ——, “sysMonitor – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-sysMonitor-SD.pdf.

[17] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, IDD 5.2.2, 2026, simpleStoreServiceOrchestration-v5.2.2-sysMonitor-AMQP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	